SOFTWARE REQUIREMENTS SPECIFICATION

for

SHARP

Smart Hostel Administration and Resource Platform

Submitted by:

1. Vaibhav Kansal (1024160097)
2. Ayush Kamboj (1024160118)
3. Ali Ekram (1024160138)
4. Riya Gupta (1024160122)

Submitted to:

Ms. Anushka

Course Instructor

UCS503P – Software Engineering Project

Thapar Institute of Engineering and Technology

Session 2026–27 ODD

October 2026

Document status: this SRS describes the planned system and its delivery phases.

Contents

1 Introduction **2**

1.1 Purpose . . . 2

1.2 Intended Audience and Reading Suggestions . . . 2

1.3 Project Scope . . . 2

1.4 Definitions, Acronyms and Abbreviations . . . 3

2 Overall Description **3**

2.1 Product Perspective . . . 3

2.2 User Classes and Characteristics . . . 3

2.3 Product Functions . . . 4

2.4 Operating Environment . . . 7

2.5 Design . . . 7

3 System Features **10**

3.1 Description and Priority . . . 10

3.2 Functional Requirements . . . 11

3.3 Implementation Requirements . . . 13

4 Other Nonfunctional Requirements **13**

4.1 Performance Requirements . . . 13

4.2 Security Requirements . . . 13

4.3 Software Quality Attributes . . . 13

4.4 Business Rules . . . 13

5 Other Requirements **14**

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the main functions, users, operating environment, quality requirements, and business rules for SHARP, a web-based hostel administration and resource platform. It serves as a reference for development, testing, and project review.

1.2 Intended Audience and Reading Suggestions

This document is intended for the project team, the course instructor, hostel stakeholders, testers, and future maintainers. Section 1 introduces the project. Section 2 describes the system and its users. Section 3 lists the system features and functional requirements. Section 4 defines the non-functional requirements, and Section 5 records additional requirements and constraints.

1.3 Project Scope

SHARP aims to digitize common hostel workflows that are otherwise managed through paper registers or informal communication. The planned system covers room allocation, leave and day-out passes, maintenance complaints, mess feedback, and laundry issue tracking.

Current prototype status. The room-allocation process has been implemented, and the Student, Parent, and Caretaker portals have been created. The existence of these portals does not mean that all workflows within them are complete. Leave/pass processing, complaint management, laundry issue tracking, mess feedback, and the remaining modules are not yet implemented and are described as planned requirements in this SRS.

Status terms. In this document, **Implemented** identifies functionality that is already built in the prototype. **MVP** (Minimum Viable Product) identifies the functionality intended for the first complete release. **Planned** identifies functionality that is not yet implemented and is intended for a later phase.

Out of scope. Payment processing, replacement of institutional student-information systems, physical access-control hardware, emergency dispatch, and a native mobile application are outside the current scope.

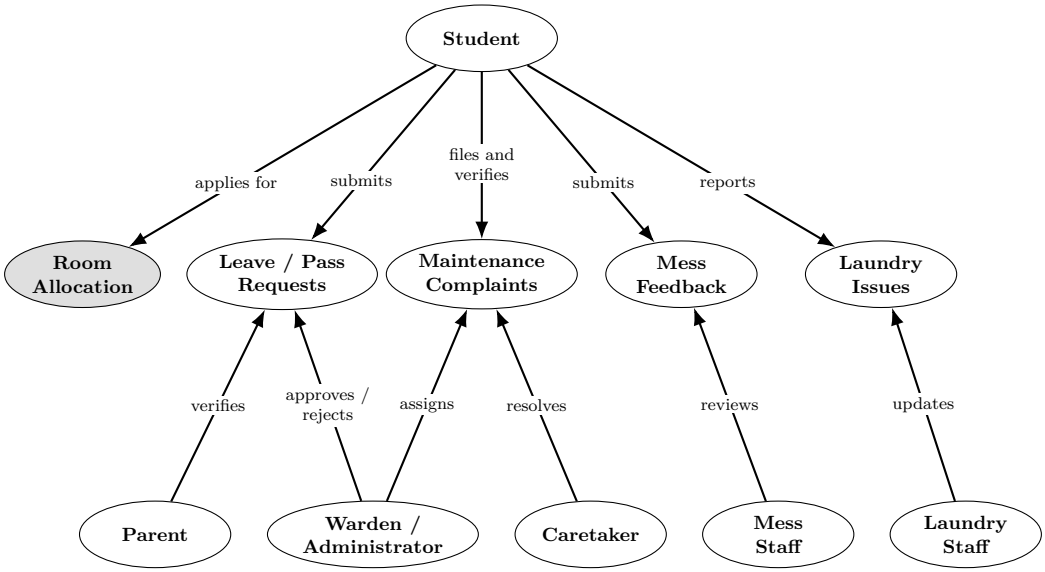


Figure 1.1: Entire work-flow

Figure 1.1 (Entire work-flow) is the overview of the project. The entities depend on one another, which gives a simple picture of the functional activities of the system. In the figure, the shaded entity (Room Allocation) is implemented and all unshaded entities are planned.

Student cycle: a student applies for a room, submits leave requests, files maintenance complaints, submits mess feedback, and reports laundry issues. The student also verifies whether a complaint has really been resolved.

Staff and parent cycle: a parent verifies leave requests of the linked student; the warden approves or rejects leave requests and assigns complaints; the caretaker resolves the assigned complaints; mess staff review mess feedback; laundry staff update laundry issues.

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
MVP	Minimum Viable Product: the smallest complete release that delivers the core hostel workflows
SLA	Service Level Agreement: the time within which a complaint must be resolved
CGPA	Cumulative Grade Point Average
API	Application Programming Interface
JWT	JSON Web Token, used by the prototype for authentication
HTTPS	Hypertext Transfer Protocol Secure
Cluster	A group of four students occupying two paired rooms that share a washroom
FR	Functional Requirement
DFD	Data Flow Diagram: shows how data moves between external entities, processes, and data stores

2 Overall Description

2.1 Product Perspective

SHARP is a browser-based web application that replaces paper registers and informal communication in hostel administration. Users interact with role-specific pages, application services process requests, and a database stores user and workflow records. The planned design separates the interface, application/API, workflow services, and persistence layers. The prototype uses Node.js/Express and MongoDB (see Section 3.3); these are implementation choices and not mandatory product requirements.

2.2 User Classes and Characteristics

SHARP has the following user classes (Figure 2.1):

- **Student:** applies for a room, submits leave requests, maintenance complaints, mess feedback, and laundry issue reports, tracks their status, and verifies complaint resolution.
- **Parent:** verifies or declines leave requests linked to their student, where required by hostel policy.
- **Warden / Administrator:** approves or rejects leave requests, assigns complaints, and oversees escalations. As the authorized administrator, this role also provisions accounts, assigns roles, and maintains approved workflow settings.

- **Caretaker:** accepts assigned maintenance work, updates progress, and records completion.
- **Mess Staff:** reviews categorized mess feedback (later phase).
- **Laundry Staff:** reviews and updates laundry issue reports (later phase).

Access to each record shall be limited by role and, where applicable, by record ownership or assignment.

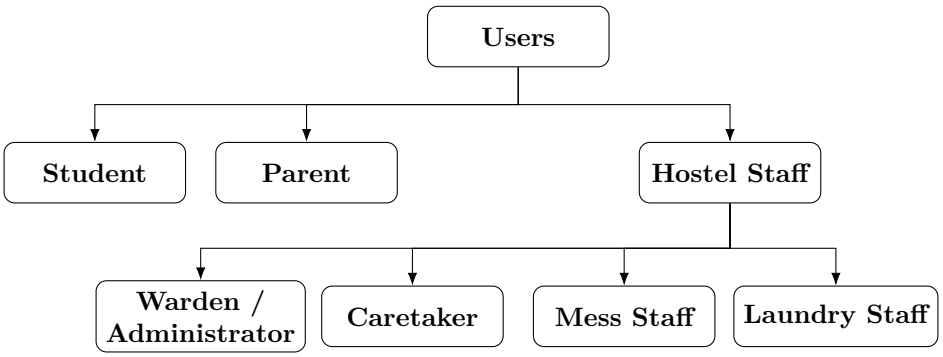


Figure 2.1: Types of users

2.3 Product Functions

The main functions of SHARP and their current status are:

1. **Portals:** the Student, Parent, and Caretaker portal interfaces have been created; workflows within them may still be incomplete. Warden/Administrator and service-staff interfaces are planned.
2. **Room allocation (Implemented):** supports cluster/roommate preferences, a cluster leader, three room preferences, and automatic allocation using cluster-average CGPA to resolve conflicts.
3. **Leave and day-out passes (MVP):** submit requests, record parent verification where required, obtain warden decisions, and issue a digital pass after all required approvals.
4. **Maintenance complaints (MVP):** submit and assign complaints, track work and SLA deadlines, and close a complaint only after student verification.
5. **Mess feedback (Planned):** submit categorized feedback and show its content to mess staff without exposing the reporter's identity in ordinary staff views.
6. **Laundry issue tracking (Planned):** report missing, damaged, incorrect, stained, or delayed items and track their handling status.
7. **Notifications and reports (Planned, optional):** provide workflow notifications and operational summaries when approved and configured.

Every user has an account with a role. Students additionally belong to a cluster for room allocation.

Data flow. Figure 2.2 is the context-level (Level 0) data flow diagram. It shows SHARP as a single process and the data exchanged with each external user class. Figure 2.3 is the Level 1 data flow diagram, which splits SHARP into six processes (1.0 to 6.0) and six data stores (D1 to D6). In both figures, rectangles are external entities, rounded rectangles are processes, open-ended rectangles are data stores, and arrows are data flows. A double-headed arrow carries two flows; in its label, $\rightarrow$ marks the flow that moves to the right and $\leftarrow$ the flow that moves to the left. In Figure 2.3, an external entity (for example the Student or the Warden / Administrator) is drawn once for each process it interacts with, only to avoid crossing lines. Of the processes, only 2.0 (room allocation) is implemented, 1.0 is partially built, and 3.0 to 6.0 are planned.

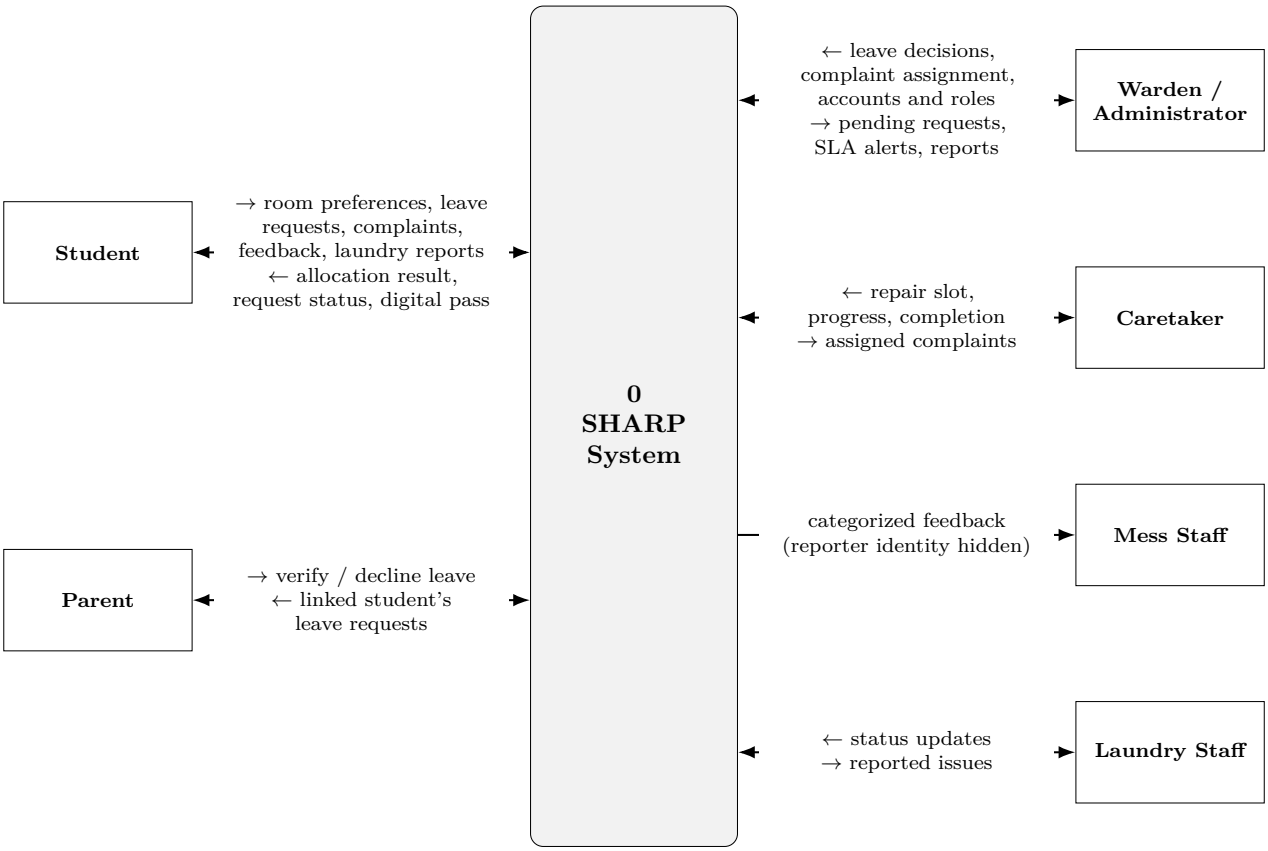


Figure 2.2: Data Flow Diagram (Level 0)

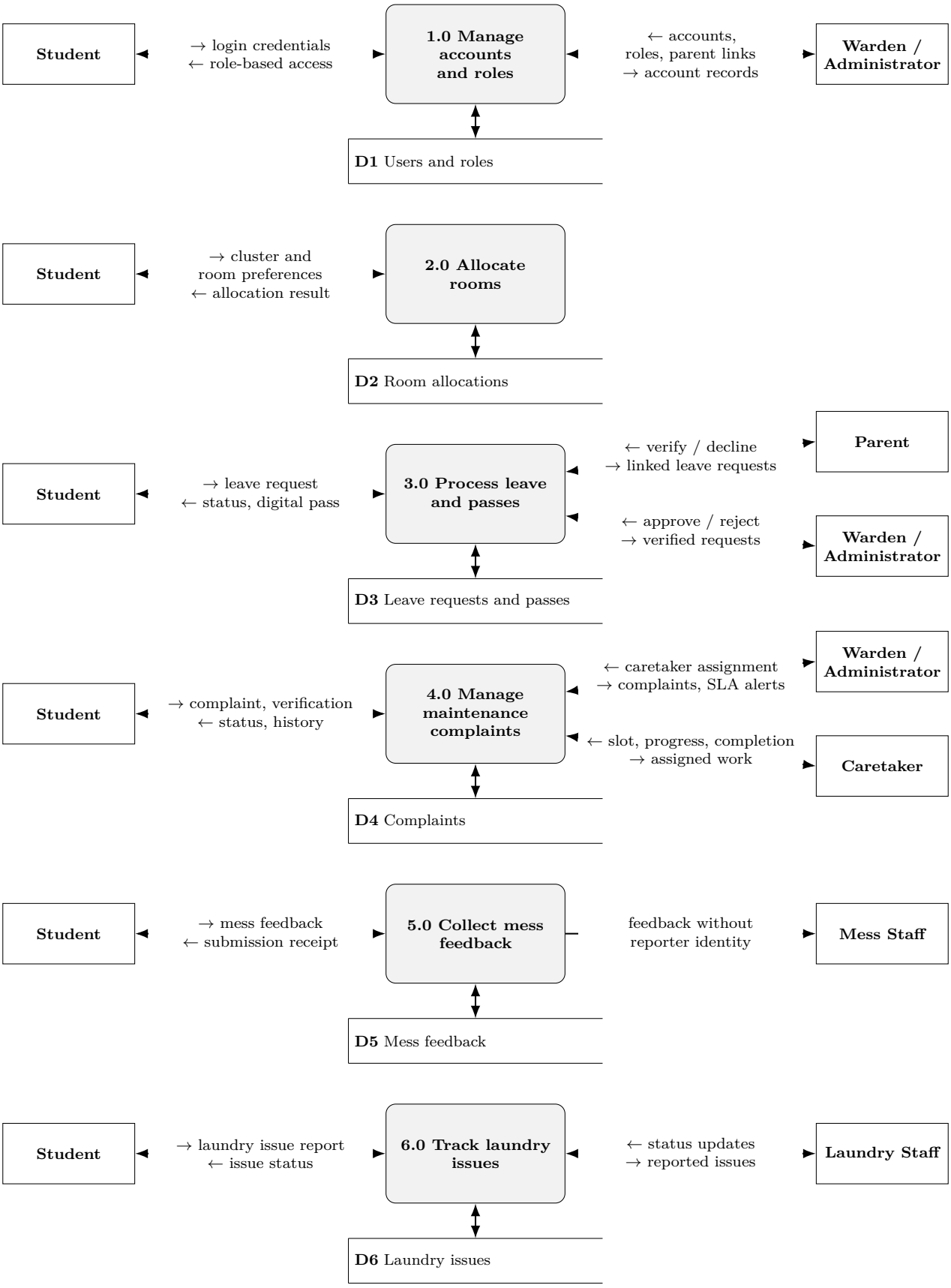


Figure 2.3: Data Flow Diagram (Level 1)

2.4 Operating Environment

The system shall be accessed through a modern web browser on a network-connected desktop or mobile device. The server shall run a web application with persistent database storage. The supported browser versions and the hosting environment shall be confirmed before deployment. Offline operation is not required.

2.5 Design

The system shall use role-specific pages and forms. Forms shall identify required fields and display validation errors. Users shall be able to view the current status of their own requests and the history permitted by their role.

Student activities have 3 stages (Figure 2.4):

- Room Allocation Process
- Service Requests
- Student Dashboard

A student first forms a cluster of four, and the cluster leader submits three room preferences. After the automatic allocation, the student can submit leave requests, complaints, mess feedback and laundry reports, and track or verify them. Every student dashboard contains the allocated room, leave status and digital pass, complaint status, mess feedback and laundry issue status.

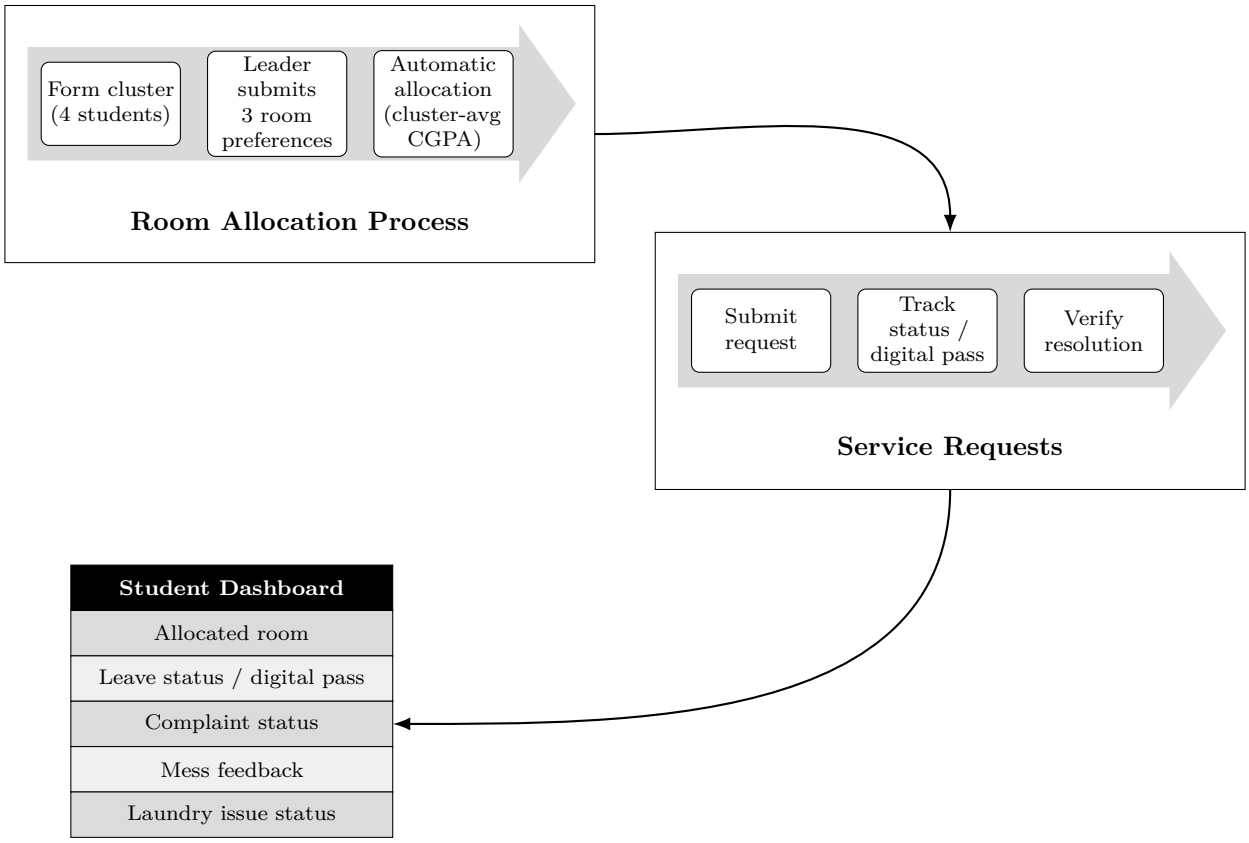


Figure 2.4: Student Activities

Parent activities (Figure 2.5) have 2 steps: view the requests of the linked student and verify or decline a leave request.

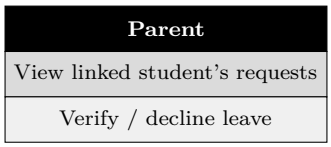


Figure 2.5: Parent Activities

Hostel staff activities have 2 types (Figure 2.6): Warden / Administrator and Caretaker. The Warden / Administrator can approve or reject leave, assign complaints, monitor SLA breaches, provision accounts and roles, and view reports. The caretaker accepts the complaints assigned by the warden, proposes a repair slot, updates progress and marks work complete.

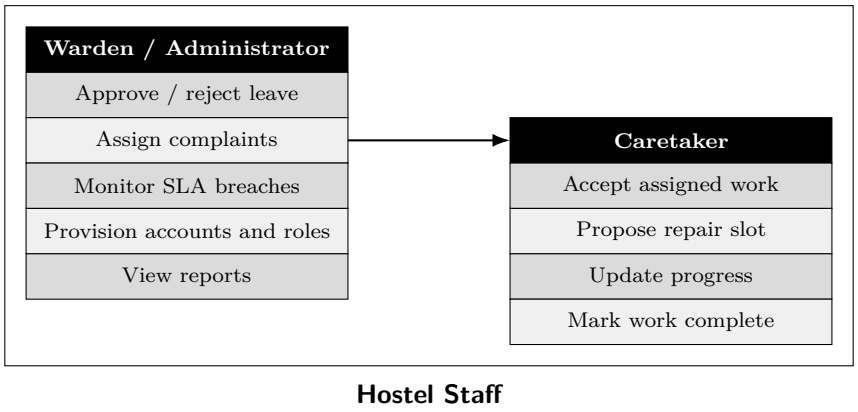


Figure 2.6: Hostel Staff Activities

Service staff have only one activity each (Figure 2.7): mess staff review categorized feedback, and laundry staff update the status of laundry issues. Both are planned for a later phase.

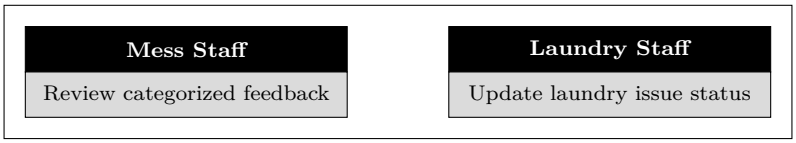


Figure 2.7: Service Staff Activities

Room allocation (implemented). Users provide roommate/cluster preferences. A cluster contains four students occupying two paired rooms that share a washroom. The cluster leader submits three room preferences, and automatic allocation uses cluster-average CGPA to resolve competing preferences. Further validation and acceptance testing should confirm edge cases and allocation fairness.

Leave/pass process (planned). Student submission, parent verification where required, warden decision, and pass issuance after all required approvals (Figure 2.8). A rejection at any stage stops the request from proceeding.

Complaint process (planned). Student submission, complaint ID and priority, caretaker assignment, repair-slot agreement (or an alternative slot proposed by the caretaker), repair, and student verification (Figure 2.9). Verified complaints are closed; a rejected resolution reopens the same complaint record. An SLA breach notifies the warden.

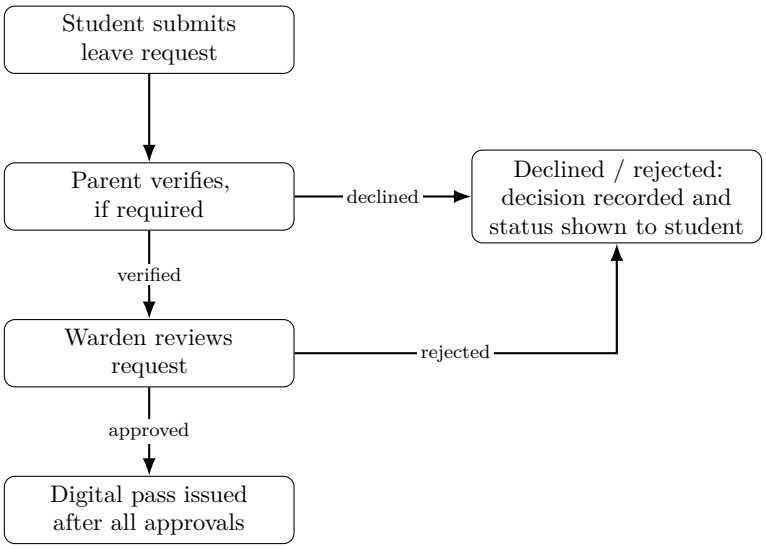


Figure 2.8: Leave/pass workflow

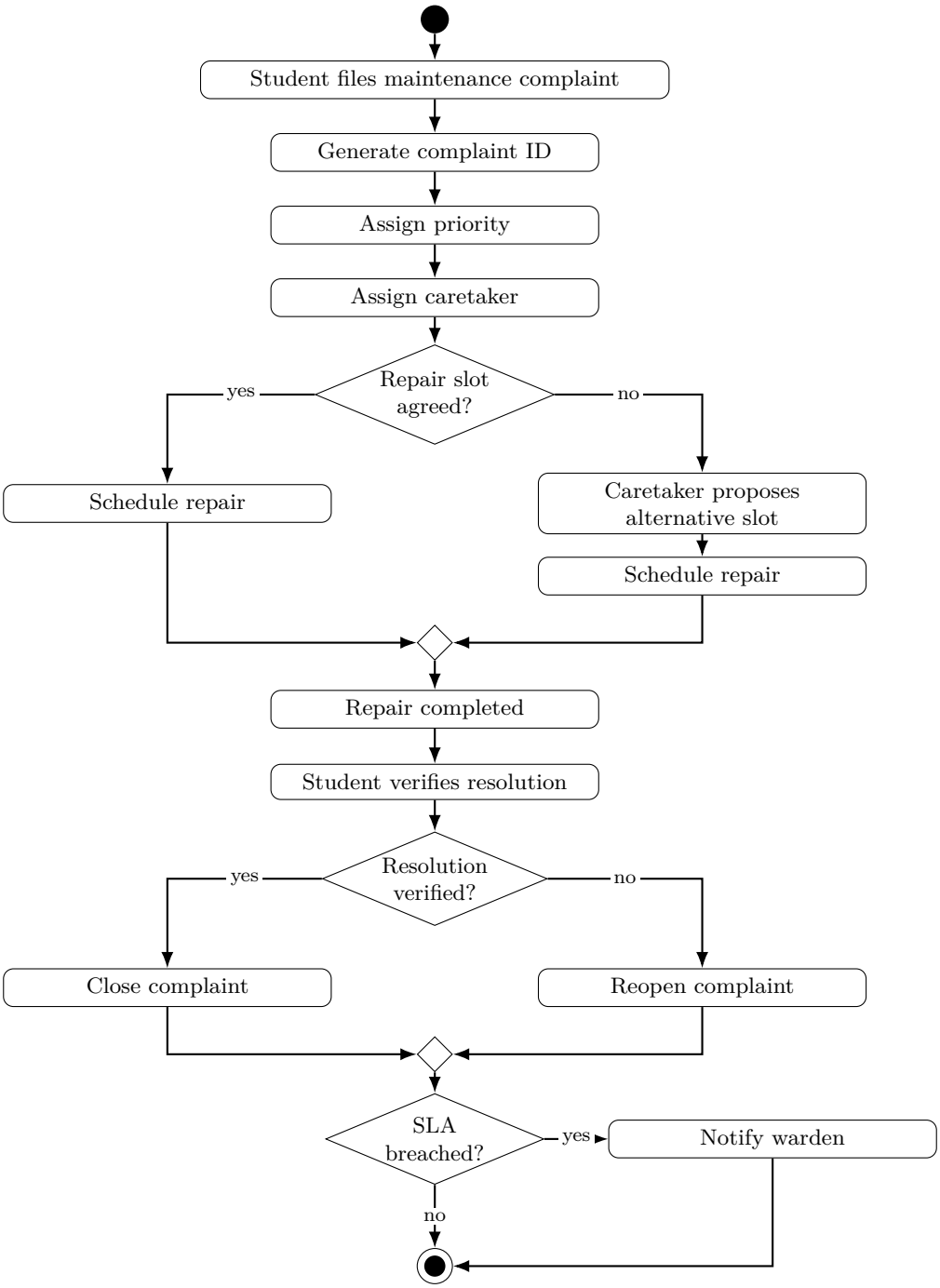


Figure 2.9: Maintenance complaint activity diagram

3 System Features

SHARP is a hostel workflow platform. Its main purpose is to move the common hostel processes from paper and informal messages into one role-controlled system.

3.1 Description and Priority

The table distinguishes the current implementation status from the delivery priority. The portal interfaces exist, but only the room-allocation process is currently implemented. MVP and Planned identify the intended delivery phases, not completed functionality.

Feature	Description	Priority	Status
Portal interfaces	Student, Parent, and Caretaker portals have been created; not all workflows are implemented.	MVP foundation	Partially built
Room allocation	Cluster/roommate preferences and CGPA-based automatic allocation.	MVP	Implemented
Authentication and roles	Secure sign-in and role/ownership-based access to protected operations.	MVP	Partially built (completeness to be verified)
Leave/pass management	Submit leave requests, record required verification and approval, and issue approved passes.	MVP	Not implemented
Maintenance complaints	Submit, assign, update, escalate, and verify complaints before closure.	MVP	Not implemented
Mess feedback	Collect categorized feedback while hiding reporter identity from ordinary mess-staff views.	Planned	Not implemented
Laundry issues	Record and track missing, damaged, incorrect, stained, or delayed items.	Planned	Not implemented
Notifications and reports	Optional workflow notifications and administrative summaries.	Planned	Not implemented

3.2 Functional Requirements

Each functional requirement has an identifier of the form **FR-*MODULE-NN***, where *MODULE* is a short code for the feature area and *NN* is a two-digit sequence number within that area. The module codes are: AUTH (authentication and access), LEAVE (leave and day-out passes), CMP (maintenance complaints), ROOM (room allocation), MESS (mess feedback), LDRY (laundry issues), NTF (notifications), and RPT (reports). Identifiers are never reused, so they can be referenced from test cases and review comments.

Authentication and role access (MVP; portal interfaces created)

FR-AUTH-01 The system shall authenticate users before granting access to protected records or actions.

FR-AUTH-02 The system shall associate each user with an authorized role.

FR-AUTH-03 The system shall check the role and, where applicable, the record ownership or assignment for each protected operation.

FR-AUTH-04 The system shall deny unauthorized access and shall not store credentials in readable form.

FR-AUTH-05 The system shall record significant workflow decisions and status changes with timestamps and the responsible authorized user where applicable.

FR-AUTH-06 Only an authorized administrator (the Warden / Administrator role) shall be able to provision accounts, assign roles, and establish parent–student links.

Leave and day-out passes (MVP; not implemented)

FR-LEAVE-01 A student shall be able to submit a leave request with the dates, times, destination, and reason required by hostel policy.

FR-LEAVE-02 The system shall validate required fields and prevent inconsistent date/time ranges.

FR-LEAVE-03 Each request shall receive a unique identifier, a submission timestamp, and a pending status.

FR-LEAVE-04 The system shall route a request for parent verification before warden approval where policy requires it.

FR-LEAVE-05 A linked parent shall be able to verify or decline a request, and the system shall preserve the decision and its timestamp.

FR-LEAVE-06 An authorized warden shall be able to approve or reject a request. A request declined at an earlier required stage shall not proceed to approval.

FR-LEAVE-07 The student and authorized reviewers shall be able to view the current status and decision history.

FR-LEAVE-08 The system shall issue a uniquely referenced digital pass only after all required approvals.

Maintenance complaints (MVP; not implemented)

FR-CMP-01 A student shall be able to submit a complaint with a category, description, and preferred repair slot.

FR-CMP-02 Each complaint shall receive a unique identifier, a filing timestamp, and a priority based on configured rules.

FR-CMP-03 An authorized warden shall be able to assign a complaint to a caretaker, and the system shall record the assignment and its time.

FR-CMP-04 The assigned caretaker shall be able to acknowledge the complaint, propose or accept a repair slot, and update progress.

FR-CMP-05 The system shall track the configured SLA deadline and make breached open complaints visible to the warden.

FR-CMP-06 A caretaker shall be able to mark work complete, but this shall not by itself close the complaint.

FR-CMP-07 The student who filed the complaint shall be able to verify or reject the claimed resolution. Verification shall close the complaint; rejection shall reopen the same record and preserve its history.

FR-CMP-08 Students and authorized staff shall be able to view the complaint status and the permitted assignment/status history.

Room allocation (Implemented)

FR-ROOM-01 The system shall allow a cluster leader to submit roommate/cluster preferences for a group of four students, according to the allocation rules.

FR-ROOM-02 The allocation process shall support three room preferences for the cluster leader.

FR-ROOM-03 When multiple clusters compete for room preferences, the system shall use cluster-average CGPA as the allocation priority rule.

FR-ROOM-04 The system shall record the resulting room allocation and make the result available to authorized users.

Mess feedback, laundry, notifications and reports (Planned)

FR-MESS-01 A student shall be able to submit categorized mess feedback.

FR-MESS-02 Authorized mess staff shall be able to review the feedback content, and ordinary mess-staff views shall not expose the reporter's identity.

FR-LDRY-01 A student shall be able to report a laundry issue and view its status.

FR-LDRY-02 Authorized laundry staff shall be able to update the handling status of a reported issue.

FR-NTF-01 Where an approved notification provider is configured, the system should notify relevant users of workflow events.

FR-RPT-01 Authorized administrative users should be able to view operational summaries

based on recorded workflow data.

3.3 Implementation Requirements

The application is being developed as a web application with a user interface, an application/API layer, workflow logic, and persistent data storage. The prototype uses a React/Vite frontend, a Node.js/Express backend, MongoDB Atlas, and JWT authentication. This is the current prototype stack; implementation status is recorded separately from the target requirements.

4 Other Nonfunctional Requirements

4.1 Performance Requirements

The following targets are sized for a pilot with a small number of users and shall be checked during testing:

- Ordinary pages and form submissions should respond within about 3 seconds for up to 20 users using the system at the same time.
- The room-allocation run for a hostel block should complete within one minute.

The test method and results shall be recorded.

4.2 Security Requirements

- Protected actions shall require authentication and server-side authorization.
- Passwords, if locally managed, shall be stored using a modern salted password-hashing algorithm; plaintext passwords shall not be stored or logged.
- Production deployment should use HTTPS.
- Sensitive records shall be visible only to authorized roles and to users with the appropriate record relationship.
- The mess-feedback reporter's identity shall be hidden from mess staff in ordinary views and service responses.

4.3 Software Quality Attributes

- **Reliability:** successful workflow changes shall persist and be visible on subsequent authorized reads.
- **Usability:** forms shall label required fields, explain validation errors, and show the current workflow status.
- **Maintainability:** workflow modules should have separable responsibilities and automated tests for critical transitions.
- **Compatibility:** the interface should support current stable versions of Chrome, Firefox, and Edge during pilot testing, with responsive layouts at widths of 360 CSS pixels and above.
- **Auditability:** important decisions, assignments, status changes, reopenings, and SLA breaches shall retain timestamps and prior history.

4.4 Business Rules

- Each request, complaint, pass, or report shall have a unique identifier and a creation timestamp.
- A digital leave pass shall be issued only after the approvals required by hostel policy.
- Completion of complaint work and closure of the complaint are separate events; closure requires student verification.
- A rejected resolution shall reopen the same complaint record rather than create a replacement

record.
- Priority rules, SLA thresholds, leave fields, and approval exceptions shall be configured according to hostel policy.
- The implemented room-allocation process uses cluster-average CGPA to resolve competing room preferences; the allocation rules should be documented and tested with representative cases.
- Access to personal records shall be restricted, and retention/deletion periods shall be agreed with institute policy before production use.

5 Other Requirements

SHARP shall be designed so that planned hostel modules can be added without unnecessarily affecting the existing room-allocation process and portal interfaces. The current prototype has the room-allocation process and the Student, Parent, and Caretaker portals; leave/pass processing, complaint management, laundry tracking, mess feedback, notifications, and reporting remain unimplemented. These modules shall be developed and tested before being described as completed.

The following shall be confirmed with the relevant stakeholders before pilot or production use: account provisioning, parent–student linking, leave policies, complaint priorities, SLA thresholds, data retention, notification configuration, and hosting/security responsibilities.